

VIP anatomy

An agent drives and observes one interface, but a reusable verification IP is more than an agent

Four deliverables

- The agent is the protocol unit, sequencer, driver, monitor, and interface around one bus
- The checker is passive protocol rules
- Coverage is covergroups and bins that measure which scenarios the VIP exercised
- Docs are the handoff, API notes, README, examples, and a self-test consumers can run
- Scoreboards often live in the env; VIPs commonly ship checker plus coverage with the agent
- A package is complete only when all four deliverables are present and consumers can

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / VIP anatomy

INTERACTIVE TOOL

VIP anatomy

Sketch a reusable **verification IP package**: **agent** + **checker** + **coverage** + **docs**.
Starter: UART VIP with all four pieces present — complete.

Starter example: UART VIP with agent + checker + coverage + docs — package COMPLETE.

Load starter example

Challenges 0 / 22 cleared

Quiz: VIP: A verification IP (VIP) package is mainly...

☐ a reusable agent + checker + coverage + docs handoff

☐ only a Makefile PHONY target

☐ a replacement for the DUT

☐ just +UVM_TESTNAME

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

agent

Drive / observe one protocol interface.

checker

Passive bus-rule watch shipped with the VIP.

coverage

Bins that prove which scenarios ran.

docs

API + examples + self-test for handoff.

Lab

SCENARIO

VIP anatomy lab starter

Real UVM literacy

```
❏ ● ● wsl - module21-vip-anatomy/examples/vip-sketch

# Track A - real Unix
# real shell session (Track A ~ VIP anatomy)

$ pwd
/mt/d/proj/designs/learning/courses/learn_uvm2017/module21-vip-anatomy

$ ls examples/vip-sketch/
vip.txt

$ cat examples/vip-sketch/vip.txt
VIP anatomy sketch (Literacy)

Verification IP package = reusable handoff unit for one protocol/interface.

Four deliverables (all required for COMPLETE):
agent    - drive / observe one interface
          - sequencer + driver + monitor + vif / seq items
checker  - passive bus-rule / protocol legality
          - orthogonal to env scoreboard (payload expect/actual)
coverage - coveragegroups / bins: which scenarios did the VIP exercise?
docs     - API notes, README, examples, self-test for consumers

Starter (COMPLETE):
UART VIP with agent + checker + coverage + docs

Incomplete examples:
missing checker + INCOMPLETE (no legality)
missing coverage + INCOMPLETE (no measurement)
missing docs    + INCOMPLETE (no handoff)
agent only      + scaffold, not a VIP package yet

Typical placement:
VIP ships: agent, checker, coverage, docs
Env often owns: scoreboard, virtual sequences, test glue

Consumer integrate flow:
1. instantiate / factory agent
2. bind / enable checker
3. sample / close coverage
4. read docs + run self-test

Common mistakes:
calling agent-only tree a "VIP"
stuffing project scoreboard into every VIP
shipping code without self-test / README
assuming compile success == package complete

Legacy reference:
learn_uvm2017_sv_verilator/module7/examples/vip/vip_sv
(agent scaffold - txn + seq + driver + agent; add checker/cov/docs for full VIP)

$ grep -n "VipAgent\\|VipDriver\\|VipTxn\\|VIP" ../../learn_uvm2017_sv_verilator/module7/examples/vip/vip_sv | head -34
2: * Module 7 Example 7.5: VIP Development (skeleton)
3: * Demonstrates how a protocol VIP might be packaged: txn + seq + agent.
6: * 1. Understand VIP (Verification IP) structure
7: * 2. Learn VIP packaging patterns
8: * 3. Understand reusable VIP components
9: * 4. Learn VIP agent structure
10: * 5. Understand VIP transaction modeling
12: * VIP STRUCTURE:
13: *   - Transaction: VipTxn (protocol-specific data)
14: *   - Sequence: VipSeq (transaction generation)
15: *   - Driver: VipDriver (protocol driving)
16: *   - Agent: VipAgent (complete VIP component)
18: * VIP PACKAGING:
31: * VipTxn: VIP transaction class
```

Real shell — VIP anatomy sketch

Pitfalls to watch

- Do not call an agent-only tree a VIP, consumers need checker, coverage, and docs too
- VIP ships protocol legality and coverage
- Do not skip the self-test in docs, integration without a smoke run fails at the consumer
- Do not forget that incomplete packages still compile
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser
- On real UVM, sketch a UART VIP tree with all four deliverables labeled
- When you are ready